

Documento de Justificación de Decisiones de Diseño

Este documento tiene como objetivo justificar las principales decisiones de diseño tomadas para la implementación de la User Story “**Inscribirme a una actividad**” en el marco del Trabajo Práctico 6 (TDD - Desarrollo Conducido por Pruebas), perteneciente a la materia **Ingeniería y Calidad de Software** de la UTN - Facultad Regional Córdoba.

1. Enfoque de desarrollo (TDD)

Se aplicó el enfoque **Red-Green-Refactor** del Desarrollo Conducido por Pruebas (TDD), donde primero se escribieron pruebas unitarias que inicialmente fallaban (Red), luego se implementó el código mínimo necesario para que las pruebas pasaran (Green), y finalmente se realizaron mejoras en la estructura del código manteniendo las pruebas exitosas (Refactor). Este enfoque permitió garantizar la calidad del código desde el inicio, asegurando que cada requerimiento estuviera cubierto por al menos una prueba automatizada y promoviendo un diseño modular y testeable.

2. Decisiones de diseño principales

A continuación, se detallan las decisiones más relevantes tomadas durante el desarrollo: **a)**

Estructura modular de archivos

Se dividió la funcionalidad en módulos independientes: **Inscripcion.py**: contiene la lógica principal para la inscripción de usuarios a actividades. **Validaciones.py**: incluye funciones de validación reutilizables que garantizan la coherencia y correctitud de los datos de entrada. **DB.py**: centraliza la conexión a la base de datos, manejo de transacciones y commit/rollback. **Tests_Inscripcion.py y Tests_Validaciones.py**: contienen las pruebas unitarias y de integración que aseguran el correcto funcionamiento de las distintas partes del sistema. Esta separación favoreció la mantenibilidad, el aislamiento de responsabilidades y la reutilización del código. **b) Manejo de transacciones y conexión a base de datos**

Se decidió implementar una capa de abstracción en **DB.py** para gestionar la conexión y las transacciones de forma controlada. Esto evita accesos directos y repetitivos a la base de datos desde múltiples módulos y facilita el control de errores mediante commit/rollback automáticos. Durante los tests, se optó por inicializar la conexión en un entorno controlado, evitando errores como NoneType object has no attribute 'cursor' mediante la creación de transacciones simuladas o mockeadas según el caso. **c) Validaciones separadas**

Las validaciones (por ejemplo, verificar cupos disponibles, existencia del usuario, y talles opcionales) fueron extraídas del flujo principal para mantener el principio de responsabilidad única (SRP). Esto permite probar las reglas de negocio de manera aislada y evita duplicar lógica en futuras funcionalidades. **d) Pruebas unitarias y de integración**

Las pruebas se estructuraron siguiendo los principios de TDD: cada nueva funcionalidad se desarrolló a partir de una prueba que definía su comportamiento esperado. Se incluyeron tanto casos exitosos como casos de error para cubrir todos los escenarios posibles. **e) Control de errores y mensajes descriptivos**

Se implementó un manejo controlado de excepciones con mensajes descriptivos para el usuario, mejorando la trazabilidad en caso de fallas y facilitando la depuración durante el desarrollo y testing.

3. Beneficios obtenidos

Las decisiones de diseño tomadas aportaron los siguientes beneficios: **Alta cobertura de pruebas**: Todas las funcionalidades implementadas cuentan con pruebas automatizadas. **Mantenibilidad**: La separación modular facilita futuras modificaciones sin afectar otras partes del sistema. **Reutilización**: Las validaciones y la gestión de base de datos son componentes reutilizables en otros módulos. **Confiabilidad**: El uso de transacciones controladas evita inconsistencias en la base de datos. **Escalabilidad**: La arquitectura modular permite incorporar nuevas funcionalidades (por ejemplo, cancelación o modificación de inscripciones) con bajo

impacto.

4. Conclusión

El enfoque TDD aplicado permitió no solo cumplir los criterios de aceptación de la User Story “Inscribirme a una actividad”, sino también lograr un diseño limpio, modular y fácilmente extensible. Las decisiones de diseño adoptadas refuerzan los principios de calidad del software, promoviendo la trazabilidad, la mantenibilidad y la robustez del sistema.